Data Analytics · Python Lab Exercise

NumPy Essentials — Class Practice 7.1

Jupyter Notebook · NumPy · Arrays · Mathematical Operations · Data Analysis

1. Scenario

SalesMetrics Co. — Weekly Revenue Analyser

You are a junior data analyst at SalesMetrics Co. Your team tracks weekly revenue figures across five regional offices. Your manager has asked you to build a Jupyter Notebook that uses NumPy to load, inspect, and analyse the data — producing key statistics and identifying high-performing regions.

Your completed notebook must:

1. Create a 1-D NumPy array of weekly revenue figures for five regions.
2. Use NumPy built-in functions to calculate descriptive statistics.
3. Perform element-wise arithmetic on the array.
4. Apply Boolean masking to filter the array based on a condition.
5. Reshape the 1-D array into a 2-D array and analyze it by rows and columns.
1. Document each step with a Markdown cell above every Code cell.

Jupyter Setup

Launch Jupyter: jupyter notebook in your terminal.
Create a new notebook — File > New > Notebook > Python 3 (ipykernel).
Rename it: DA_Exercise_7_1.ipynb
Run each Code cell with Shift+Enter. Add Markdown cells with Esc → M.
Save frequently with Ctrl+S.

2. Requirements

Your notebook must satisfy all six requirements below. Each requirement maps to one or more Code cells, each preceded by a Markdown cell.

1 Req

Notebook Header — Markdown Cell

Add a Markdown cell at the top of the notebook containing:
Data Analytics — Exercise 7.1: NumPy Essentials
Your Name | Date
A brief (2-sentence) description of what the notebook does.

2 Req

Import NumPy and Create the Revenue Array

import numpy as np — always the first Code cell.
Create a 1-D array called revenue using np.array() containing these five floats:
[52_400.0, 38_750.0, 61_200.0, 47_900.0, 55_650.0]
Print the array and its dtype and shape attributes.

3 Req

Descriptive Statistics

Use NumPy functions to compute and print:
Total revenue → np.sum()
Mean revenue → np.mean()
Median revenue → np.median()
Std deviation → np.std()
Min / Max values → np.min() / np.max()
Format every result to 2 decimal places using an f-string.

4 Req

Element-Wise Arithmetic

Apply a 5% performance bonus to every region — multiply the array by 1.05.
Store the result in a new array called revenue_bonus.
Print both the original and bonus arrays on separate labelled lines.
Calculate and print the bonus amount per region: revenue_bonus - revenue.

5 Req

Boolean Masking — Filter High-Performing Regions

Create a Boolean mask: above_avg = revenue > np.mean(revenue)
Print the mask (array of True/False).
Use the mask to filter and print only the above-average revenue values.
Print how many regions beat the average using np.sum(above_avg).

6 Req

Reshape to 2-D and Analyse

Reshape revenue into a (5, 1) column array and print it.
Create a 2-D array called q_data of shape (2, 5) by stacking revenue twice using np.vstack().
Print q_data.
Use axis=1 to print the row totals and axis=0 to print the column (region) means.
Write a Markdown Summary cell interpreting the 2-D results in plain English.

3. Hints

NumPy quick reference — compare the given example vs your task

Given example (temperatures)	Your task (revenue)
`import numpy as np`	`import numpy as np`

`temps = np.array([22.5, 18.0, 30.2, 25.7, 19.4])`	`revenue = np.array([52_400.0, 38_750.0, ...])`
`print(temps.dtype, temps.shape)`	`print(revenue.dtype, revenue.shape)`
`np.mean(temps)  # → 23.16`	`np.mean(revenue)  # → 51_180.0`
`temps * 1.10  # 10% increase`	`revenue * 1.05  # 5% bonus`
`temps[temps > 22]  # Boolean mask`	`revenue[revenue > np.mean(revenue)]`
`temps.reshape(5,1)`	`revenue.reshape(5,1)`
`np.vstack([temps, temps])  # shape (2,5)`	`np.vstack([revenue, revenue])`

1 Hint

Import convention — always use np as the alias

`import numpy as np`

Using np as the alias is the universal convention in data analytics.

`After this import, every NumPy function is accessed as np.function().`

2 Hint

Array attributes you must print

`arr = np.array([52_400.0, 38_750.0, 61_200.0, 47_900.0, 55_650.0])`

print(arr.dtype) # float64 — NumPy infers the data type

print(arr.shape) # (5,) — a tuple showing dimensions

Python underscores in numbers (52_400.0) are just readability — no effect on the value.

3 Hint

Formatting statistics with f-strings

Use :.2f inside curly braces to show exactly two decimal places:

```
print(f'Mean revenue : ${np.mean(revenue):,.2f}')
```

The comma inside :,.2f adds thousand separators: $51,180.00

Apply this format to every printed statistic in Requirement 3.

4 Hint

Element-wise arithmetic and difference

revenue_bonus = revenue * 1.05 # multiplies every element by 1.05

bonus_amount = revenue_bonus - revenue # element-wise subtraction

NumPy applies the operation to every element simultaneously — no loop needed.

5 Hint

Boolean masking — how it works

`above_avg = revenue > np.mean(revenue)  # produces: [True False True ...]`

revenue[above_avg] # returns only the values where the mask is True

`np.sum(above_avg) counts how many True values are in the mask.`

Remember: True == 1 and False == 0 in NumPy's integer arithmetic.

4. Solution

> *Attempt all six requirements in your notebook before reading the solution below. Use the hints and the example table as your guide.*

DA_Exercise_7_1.ipynb — complete notebook cell sequence

CELL 1 — Markdown cell (Esc → M before typing)

Markdown — *Notebook header*

```
# Data Analytics — Exercise 7.1: NumPy Essentials
## Your Name  |  Date: 2025-01-15

This notebook uses NumPy to analyse weekly revenue data for five
regional offices. It covers array creation, descriptive statistics,
arithmetic operations, Boolean masking, and 2-D reshaping.
```

CELL 2 — Markdown cell

Markdown — *Section heading — import and data*

```
## 1. Import NumPy and Create the Revenue Array
We import NumPy and create a 1-D array of weekly revenue figures.
```

CELL 3 — Code cell (Shift+Enter to run)

In [ ]: *Requirement 1 & 2 — import NumPy and create array*

```
# Requirement 1 — import NumPy with standard alias
import numpy as np

# Requirement 2 — create 1-D revenue array (5 regions)
revenue = np.array([52_400.0, 38_750.0, 61_200.0, 47_900.0, 55_650.0])

print(f'Array  : {revenue}')
print(f'dtype  : {revenue.dtype}')
print(f'shape  : {revenue.shape}')
```

Out [ ]: **output**

```
Array  : [52400. 38750. 61200. 47900. 55650.]
dtype  : float64
shape  : (5,)
```

CELL 4 — Markdown cell

Markdown — *Section heading — statistics*

```
## 2. Descriptive Statistics
NumPy built-in functions compute summary statistics in one line each.
```

CELL 5 — Code cell

In [ ]: *Requirement 3 — descriptive statistics*

```
print('── Revenue Statistics ──────────────────────')
print(f'Total      : ${np.sum(revenue):>12,.2f}')
print(f'Mean       : ${np.mean(revenue):>12,.2f}')
print(f'Median     : ${np.median(revenue):>12,.2f}')
print(f'Std Dev    : ${np.std(revenue):>12,.2f}')
print(f'Minimum    : ${np.min(revenue):>12,.2f}')
print(f'Maximum    : ${np.max(revenue):>12,.2f}')
```

Out [ ]: output

```
── Revenue Statistics ──────────────────────
Total      :   $256,900.00
Mean       :    $51,380.00
Median     :    $52,400.00
Std Dev    :     $7,849.36
Minimum    :    $38,750.00
Maximum    :    $61,200.00
```

CELL 6 — Markdown cell

Markdown — *Section heading — arithmetic*

```
## 3. Element-Wise Arithmetic — 5% Bonus
Multiplying a NumPy array by a scalar applies the operation to every element.
```

CELL 7 — Code cell

In [ ]: *Requirement 4 — element-wise arithmetic*

```
revenue_bonus = revenue * 1.05   # 5% bonus — element-wise
bonus_amount  = revenue_bonus - revenue


print('Original Revenue  :', revenue)
print('Revenue + 5% Bonus:', revenue_bonus)
print('Bonus Amount      :', bonus_amount)
```

Out [ ]: output

```
Original Revenue  : [52400. 38750. 61200. 47900. 55650.]
Revenue + 5% Bonus: [55020.  40687.5  64260.  50295.  58432.5]
Bonus Amount      : [2620.   1937.5  3060.   2395.   2782.5]
```

CELL 8 — Markdown cell

Markdown — *Section heading — Boolean masking*

```
## 4. Boolean Masking — Above-Average Regions
A Boolean mask filters the array to show only values meeting a condition.
```

CELL 9 — Code cell

In [ ]: *Requirement 5 — Boolean masking*

```
avg         = np.mean(revenue)
above_avg   = revenue > avg    # produces Boolean array

print(f'Mean          : ${avg:,.2f}')
print('Mask (T/F)    :', above_avg)
print('Above average :', revenue[above_avg])
print(f'Regions above : {np.sum(above_avg)} of {len(revenue)}')
```

Out [ ]: output

```
Mean          : $51,380.00
Mask (T/F)    : [ True False  True False  True]
Above average : [52400. 61200. 55650.]
Regions above : 3 of 5
```

CELL 10 — Markdown cell

Markdown — *Section heading — 2-D arrays*

```
## 5. Reshape and 2-D Analysis
Reshaping allows us to view the data as a matrix and compute axis-based stats.
```

CELL 11 — Code cell

In [ ]: *Requirement 6 — reshape and 2-D analysis*

```
# Reshape 1-D array to column vector (5 rows × 1 column)
col_vec = revenue.reshape(5, 1)
print('Column vector (5,1):\n', col_vec)

# Stack revenue twice → shape (2, 5) representing 2 weeks
q_data = np.vstack([revenue, revenue * 1.05])  # Week 1, Week 2
print('\n2-D array (2 weeks × 5 regions):\n', q_data)

# Row totals (sum across axis=1 — across columns)
print(f'\nWeekly totals   : {np.sum(q_data, axis=1)}')
# Column means (average across axis=0 — across rows)
print(f'Region avg (2wk): {np.mean(q_data, axis=0)}')
```

Out [ ]: output

```
Column vector (5,1):
 [[52400.] [38750.] [61200.] [47900.] [55650.]]

2-D array (2 weeks × 5 regions):
 [[52400.  38750.  61200.  47900.  55650. ]
  [55020.  40687.5 64260.  50295.  58432.5]]

Weekly totals   : [256900.  269695.]
Region avg (2wk): [53710.  39718.75 62730.  49097.5  57041.25]
```

CELL 12 — Markdown cell

Markdown — *Summary interpretation*

```
## Summary

- The five regions generated a **combined total of $256,900** in Week 1.
- Region 3 ($61,200) is the top performer; Region 2 ($38,750) is the lowest.
- **3 of 5 regions** exceeded the mean of $51,380 — the team is above median.
- Applying the 5% bonus raises the Week 2 total to **$269,695**.
```

5. Expected Output

With the revenue values [52400.0, 38750.0, 61200.0, 47900.0, 55650.0], running Kernel > Restart & Run All produces the following output across all Code cells:

Notebook Output

```
── Cell 3 ──────────────────────────────────────────
Array  : [52400. 38750. 61200. 47900. 55650.]
dtype  : float64
shape  : (5,)

── Cell 5 ──────────────────────────────────────────
── Revenue Statistics ──────────────────
Total      :   $256,900.00
Mean       :    $51,380.00
Median     :    $52,400.00
Std Dev    :     $7,849.36
Minimum    :    $38,750.00
Maximum    :    $61,200.00

── Cell 7 ──────────────────────────────────────────
Original Revenue  : [52400. 38750. 61200. 47900. 55650.]
Revenue + 5% Bonus: [55020.  40687.5 64260. 50295. 58432.5]
Bonus Amount      : [2620.  1937.5  3060.  2395.  2782.5]

── Cell 9 ──────────────────────────────────────────
Mean          : $51,380.00
Mask (T/F)    : [ True False  True False  True]
Above average : [52400. 61200. 55650.]
Regions above : 3 of 5

── Cell 11 ─────────────────────────────────────────
Column vector (5,1):
 [[52400.] [38750.] [61200.] [47900.] [55650.]]
2-D array (2 weeks × 5 regions): shape (2,5)
Weekly totals   : [256900.  269695.]
Region avg (2wk): [53710.  39718.75 62730. 49097.5 57041.25]
```

Cell breakdown — what each cell produces

Cell	Type	Requirement	Key Output
1	Markdown	Header	Formatted notebook title and description
2	Markdown	Intro	Section heading for import/array
3	Code	Req 2	Array, dtype float64, shape (5,)
4	Markdown	Intro	Section heading for statistics
5	Code	Req 3	6 statistics each formatted to 2dp
6	Markdown	Intro	Section heading for arithmetic
7	Code	Req 4	Original, bonus, and difference arrays
8	Markdown	Intro	Section heading for Boolean masking
9	Code	Req 5	Boolean mask, filtered values, count
10	Markdown	Intro	Section heading for 2-D analysis
11	Code	Req 6	col_vec, q_data, row totals, col means
12	Markdown	Summary	Written interpretation of 2-D results

6. NumPy Quick Reference

The table below maps every NumPy concept used in this exercise to its syntax and purpose.

Concept	Syntax used in this exercise	What it does
Create 1-D array	`np.array([v1, v2, v3, ...])`	Converts a Python list to a NumPy ndarray
Data type	`arr.dtype`	Shows the element type (float64, int32 etc.)
Dimensions	`arr.shape`	Returns a tuple: (5,) = 1-D with 5 elements
Sum	`np.sum(arr)`	Adds all elements; axis=1 sums rows
Mean	`np.mean(arr)`	Arithmetic average of all elements
Median	`np.median(arr)`	Middle value when sorted
Std deviation	`np.std(arr)`	Spread of values around the mean

Min / Max	`np.min(arr) / np.max(arr)`	Smallest / largest element
Element multiply	`arr * 1.05`	Multiplies every element by the scalar
Element subtract	`arr_b - arr_a`	Subtracts element-by-element
Boolean mask	`arr > np.mean(arr)`	Produces True/False array
Mask filter	`arr[mask]`	Returns only elements where mask is True
Count True	`np.sum(mask)`	Counts True values (True == 1)
Reshape	`arr.reshape(5, 1)`	Changes shape without changing data
Stack vertically	`np.vstack([a, b])`	Stacks two arrays → new row each
Row totals	`np.sum(arr2d, axis=1)`	Sums across columns for each row
Column means	`np.mean(arr2d, axis=0)`	Averages across rows for each column

7. Extension Challenges

Once your base notebook runs correctly with Kernel > Restart & Run All, try these extensions:

- Add a Code cell that uses np.argmax(revenue) to find and print the index of the top-performing region. Combine with a region names list to print the region's name.
- Add a Code cell that uses np.linspace(38_750, 61_200, 5) to create a target array of evenly spaced revenue goals. Print the gap between each target and the actual revenue.
- Create a 3-D array of shape (4, 5, 1) using np.tile(revenue, (4,1,1)) representing 4 weeks. Print the overall mean and the shape.
- Use np.percentile(revenue, [25, 50, 75]) to compute the quartiles and print a boxplot-style summary (Q1, Median, Q3, IQR = Q3 - Q1).
- Add a Code cell that uses np.where(revenue > np.mean(revenue), 'High', 'Average') to create a string label array and print it alongside the original revenue values.
- Sort the revenue array with np.sort() and use np.argsort() to find the original position of each region after sorting. Print a ranked region table.